



UNIVERSIDAD TECNOLÓGICA DEL ORIENTE

Facultad de Ingeniería · Programa de Ingeniería de Software virtual

Asignatura: Electiva 2 Inteligencia Artificial Avanzada

Diseño de Algoritmos de Aprendizaje Profundo: Predicción de Precios de autos usados

*Caso de estudio: Predicción de precios de automóviles usados, paquete de datos
personalizable*

Unidad I

Profesor: Carlos Carrascal Avendaño

Elaborado por:

Juan camilo Gonzalez Ortiz

26/07/2026

Resumen

Este trabajo presenta el diseño, la implementación y la evaluación de un algoritmo de aprendizaje profundo, una red neuronal para estimar el valor de vehículos usados a partir de características relevantes.

La aplicación cuenta con una vista detallada desarrollada para Windows y Linux utilizando diferentes herramientas como lo son react, electron, zustand, axios, y entre otras con el fin de que el usuario, aunque requiera conocimientos previos para usarla aun le resulte intuitivo su uso.

El modelo cuenta con 2 capas ocultas (64 y 32 neuronas), una capa de salida y usa LeakyReLU para su activación, además para regularización se utiliza Dropout, BatchNorm1d, Weight Decay, Gradient Clipping y Early data handling para garantizar el correcto funcionamiento de los datos y evitar el overfitting ya que se manejan datos de diferentes incluyendo datos en texto lo que hace que el modelo llegue a trabajar con valores que van desde 1 o 0 (ha tenido o no accidentes) hasta 50.000 (el kilometraje), la aplicación es completamente dinámica ya que usando el modelo permite la edición de diferentes características que pueden llegar a modificar el R^2 score que se puede conseguir, de la misma manera se le permite al usuario ingresar sus conjuntos de prueba en formato csv y así podrá obtener los datos adaptados a su medida

1. Identificación y Justificación del Problema

1.1.Problemática

La determinación del precio adecuado de un vehículo usado representa un desafío tanto para compradores como para vendedores debido a la gran cantidad de factores que influyen en su valor comercial, como el año de fabricación, el kilometraje, la marca, el modelo y el historial

de accidentes. La ausencia de una referencia objetiva puede ocasionar valoraciones inexactas, dificultando la toma de decisiones durante una negociación.

1.2. Justificación

En el mercado de vehículos usados es común que los compradores no cuenten con el conocimiento suficiente para determinar si el precio solicitado por un vehículo es coherente con sus características. Esta situación genera una asimetría de información que puede conducir a decisiones de compra poco acertadas o a pagar un precio superior al valor estimado del mercado.

La aplicación propuesta utiliza una red neuronal entrenada con un conjunto de datos para estimar el precio de venta de un vehículo a partir de sus características principales. De esta manera, el sistema proporciona una referencia objetiva que puede apoyar el proceso de negociación y la toma de decisiones, reduciendo la dependencia de valoraciones subjetivas y aumentando la confianza del usuario al momento de comprar o vender un vehículo usado.

Para evitar problemas relacionados con la moneda o el país de procedencia de los vehículos en cuestión el usuario puede ingresar su propio conjunto de datos personalizados los cuales podrá usar para entrenar el modelo de forma que será válido siempre que cumpla con las características necesarias para que el modelo procese dichos datos, de esta manera se amplía el alcance de la aplicación.

2. Revisión Bibliográfica

La presente revisión se fundamenta en trabajos relacionados con el aprendizaje profundo, el entrenamiento de redes neuronales y las técnicas utilizadas para mejorar su capacidad de generalización en problemas de regresión. Estos conceptos sustentan las decisiones adoptadas durante el desarrollo del modelo para la estimación del precio de vehículos usados.

2.1. Redes neuronales para problemas de regresión

Las redes neuronales profundas constituyen una de las principales herramientas para resolver problemas de regresión debido a su capacidad para modelar relaciones no lineales entre múltiples variables de entrada y una variable objetivo. Según Sarker (2021), el aprendizaje profundo permite aproximar funciones complejas mediante arquitecturas multicapa, lo que facilita la construcción de modelos capaces de identificar patrones presentes en conjuntos de datos tabulares. Asimismo, Janiesch et al. (2021) explican que este tipo de modelos resulta especialmente útil cuando las relaciones entre las variables no pueden describirse adecuadamente mediante técnicas lineales tradicionales.

En el presente trabajo se implementa un perceptrón multicapa (MLP) para estimar el precio de vehículos usados a partir de variables como el año de fabricación, el kilometraje, la marca, el modelo y el historial de accidentes. Para el entrenamiento se emplea la función de pérdida Mean Squared Error (MSE), ampliamente utilizada en problemas de regresión por penalizar con mayor intensidad los errores de mayor magnitud, favoreciendo así una mejor aproximación entre los valores predichos y los valores reales. Adicionalmente, el optimizador Adam se utiliza para actualizar los pesos de la red, ya que combina las ventajas de Momentum y RMSProp, permitiendo una convergencia rápida y estable durante el entrenamiento.

2.2. Técnicas de preprocesamiento y regularización

El desempeño de una red neuronal depende no solo de su arquitectura, sino también de la calidad del preprocesamiento y de las estrategias empleadas para evitar el sobreajuste. Sarker (2021) destaca que la normalización de las variables de entrada favorece la estabilidad del

entrenamiento y mejora el comportamiento de los algoritmos de optimización. Por esta razón, en este proyecto se utiliza StandardScaler para normalizar tanto las variables de entrada como la variable objetivo, mientras que las variables categóricas son transformadas mediante One-Hot Encoding, permitiendo que la red procese adecuadamente información textual como la marca y el modelo del vehículo.

Para mejorar la capacidad de generalización del modelo, se incorporan diversas técnicas de regularización. Entre ellas se encuentra Dropout, que desactiva aleatoriamente un porcentaje de neuronas durante el entrenamiento para reducir la dependencia entre ellas; Batch Normalization, que estabiliza la distribución de las activaciones internas de la red; Weight Decay (regularización L2), que penaliza pesos excesivamente grandes; y Gradient Clipping, utilizado para limitar la magnitud de los gradientes y evitar problemas asociados con gradientes explosivos. En conjunto, estas técnicas contribuyen a obtener un entrenamiento más estable y a reducir el riesgo de sobreajuste, favoreciendo que el modelo mantenga un buen desempeño sobre datos no vistos previamente.

3. Diseño del Algoritmo

El algoritmo desarrollado implementa un modelo de aprendizaje supervisado basado en una red neuronal multicapa (MLP) para estimar el precio de vehículos usados. Inicialmente, el sistema valida que el archivo CSV contenga las columnas requeridas y elimina los registros con valores faltantes para garantizar la calidad de los datos. Posteriormente, las variables numéricas son normalizadas mediante StandardScaler, mientras que las variables categóricas (marca y modelo) son transformadas utilizando One-Hot Encoding, permitiendo que la red neuronal procese correctamente la información textual.

La arquitectura de la red está compuesta por dos capas ocultas completamente conectadas con funciones de activación LeakyReLU, acompañadas de Batch Normalización y Dropout para

mejorar la estabilidad del entrenamiento y reducir el riesgo de sobreajuste. El modelo es entrenado utilizando el optimizador Adam y la función de pérdida Mean Squared Error (MSE), incorporando además técnicas como Weight Decay y Gradient Clipping para favorecer una convergencia estable. Durante cada época se calculan métricas como MSE, MAE, RMSE y el coeficiente de determinación R^2 , permitiendo evaluar el desempeño del modelo tanto en los datos de entrenamiento como en los de prueba. Finalmente, el modelo entrenado y los objetos de preprocesamiento son almacenados para realizar futuras predicciones sobre nuevos vehículos sin necesidad de volver a entrenar la red varias veces en una misma sesión

Por otro lado, la aplicación también incluye una interfaz estilizada e intuitiva desarrollada en react y electron con la cual poder realizar las diferentes configuraciones para proceder con el entrenamiento del modelo.

3.1. Adquisición y procesamiento de los datos

Los datos utilizados por el modelo son proporcionados por el usuario mediante un archivo en formato CSV. Antes de iniciar el entrenamiento, el sistema verifica que el conjunto de datos contenga las columnas requeridas (Año, Kilometraje, Marca, Modelo, Ha_tenido_accidentes y Precio_venta) y elimina los registros con valores faltantes para garantizar la calidad de la información. Posteriormente, las variables numéricas son normalizadas utilizando StandardScaler, mientras que las variables categóricas son transformadas mediante One-Hot Encoding, permitiendo que la red neuronal procese correctamente la información textual. Finalmente, el conjunto de datos se divide en datos de entrenamiento y prueba, y se convierte en tensores de PyTorch para ser utilizado durante el proceso de aprendizaje del modelo, cabe aclarar que el usuario determina el porcentaje de datos que se usara para el conjunto de prueba

3.2.Arquitectura

Al revisar la literatura ya mencionada y por medio de prueba y error se optó por las siguientes decisiones de diseño:

Figura 1 (tabla). Arquitectura de la red neuronal.

Capa	Tipo	Número de neuronas	Función / Parámetros
Entrada	Entrada	Variable	Recibe las variables procesadas del vehículo (Año, Kilometraje, Accidentes, Marca y Modelo).
Oculto 1	Linear	64	Transformación lineal de las características de entrada.
	BatchNorm1d	64	Normaliza las activaciones para estabilizar el entrenamiento.
	LeakyReLU	64	Pendiente negativa = 0.1.
	Dropout	64	Probabilidad = 0.2.
Oculto 2	Linear	32	Reduce la dimensionalidad aprendiendo representaciones más compactas.
	BatchNorm1d	32	Mantiene estables las activaciones internas.
	LeakyReLU	32	Pendiente negativa = 0.1.
	Dropout	32	Probabilidad = 0.2.
Salida	Linear	1	Produce el precio estimado del vehículo.

4. Implementación y Pruebas

La aplicación fue desarrollada como una aplicación de escritorio utilizando Electron y React para el frontend, mientras que el backend fue implementado con FastAPI, estableciendo la comunicación entre ambos mediante peticiones HTTP. Para el entrenamiento del modelo se utilizó PyTorch, acompañado de bibliotecas como Scikit-learn, Pandas y NumPy para el preprocesamiento y manipulación de los datos. Adicionalmente, se emplearon herramientas como Zustand para la gestión del estado global de la aplicación, Recharts para la visualización gráfica del entrenamiento y Tailwind CSS para el diseño de la interfaz.

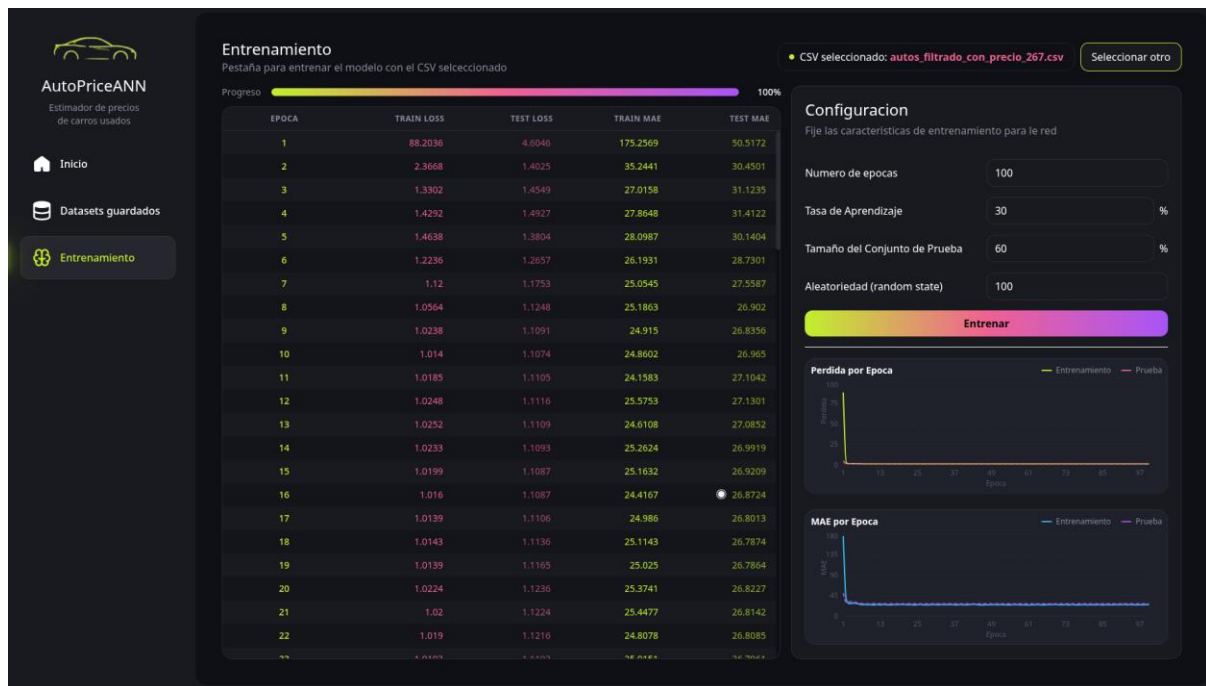
Previo al desarrollo de la aplicación se realizó un análisis de diferentes conjuntos de datos disponibles en Kaggle relacionados con la compra y venta de vehículos usados. Aunque se encontraron varios datasets, ninguno satisfacía completamente las necesidades del proyecto, por lo que se tomó como base el conjunto de datos Vehicle Dataset from CarDekho. A partir de este se generaron diferentes versiones modificadas con apoyo de herramientas de inteligencia artificial, permitiendo realizar múltiples pruebas sobre el comportamiento del modelo bajo distintos escenarios.

Durante esta etapa también se identificó que el rendimiento del modelo dependía en gran medida del origen de los datos utilizados para el entrenamiento. Aspectos como la moneda, el país de procedencia y las condiciones particulares del mercado automotriz influyen directamente en los precios de los vehículos, haciendo que un modelo entrenado con datos de un país no produzca necesariamente buenas predicciones en otro. Debido a ello, la aplicación fue diseñada para permitir que cada usuario cargue su propio archivo CSV y configure el entrenamiento de acuerdo con la información correspondiente a su contexto.

El desarrollo del modelo requirió diversas pruebas y ajustes antes de obtener resultados satisfactorios. En las primeras versiones se presentaron inconvenientes relacionados con la

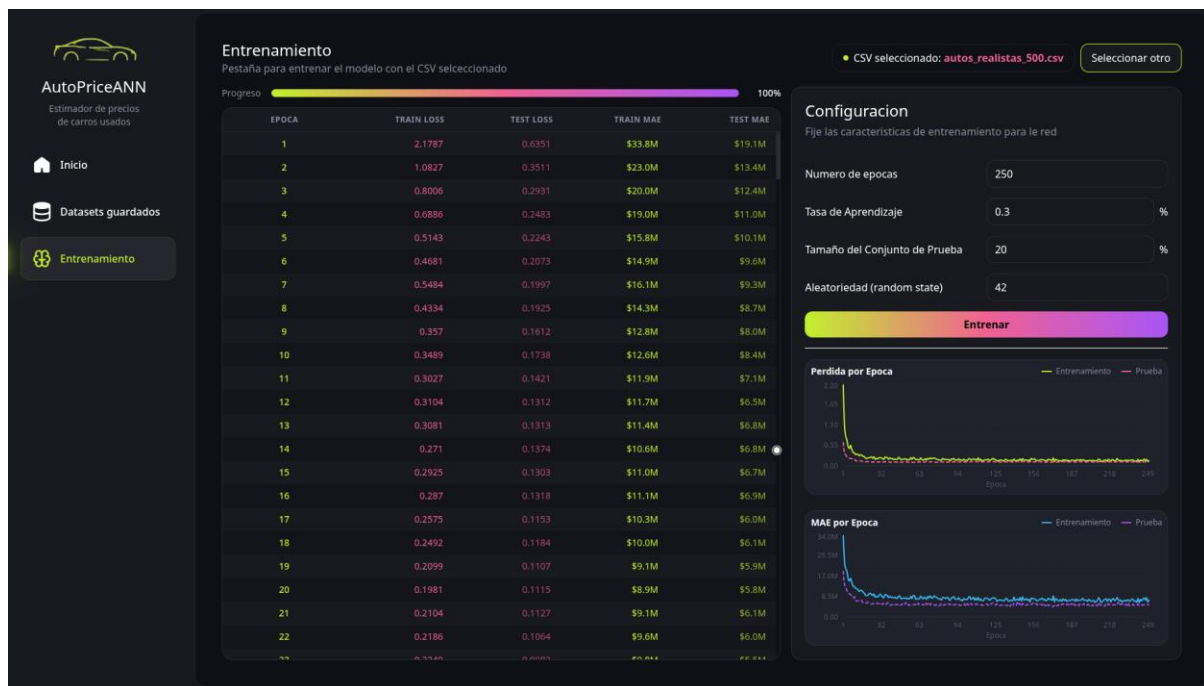
configuración del entrenamiento, el preprocesamiento de los datos y la arquitectura de la red neuronal, lo que ocasionaba pérdidas elevadas y predicciones poco precisas. A partir de estos resultados se realizaron diferentes modificaciones hasta obtener un entrenamiento más estable y una mejor capacidad de generalización.

Figura 2. Primeros resultados del entrenamiento.



En la Figura 2 se observa uno de los primeros entrenamientos realizados, donde la red neuronal aún no lograba converger adecuadamente. Las pérdidas y las métricas de error evidenciaban que el modelo no estaba aprendiendo correctamente la relación entre las características del vehículo y su precio de venta, por lo que fue necesario realizar ajustes tanto en la arquitectura como en la configuración del entrenamiento.

Figura 3. Resultados del entrenamiento después de los ajustes.



La Figura 3 presenta el comportamiento del modelo después de aplicar las mejoras realizadas durante el desarrollo. En este caso puede observarse que, aunque no con el mismo csv o configuración se presenta un entrenamiento considerablemente más estable, con una disminución progresiva de la pérdida y de las métricas de error tanto en el conjunto de entrenamiento como en el de prueba, indicando que el modelo logró aprender patrones representativos del conjunto de datos sin evidenciar un sobreajuste significativo y esta vez usando una configuración más normativa y eficiente.

Una vez verificado que el proceso de entrenamiento se realizaba correctamente y que el modelo presentaba un comportamiento estable, se procedió a evaluar su rendimiento mediante diferentes métricas de regresión. Entre ellas se utilizaron el Error Absoluto Medio (MAE), que representa el error promedio de las predicciones; la Raíz del Error Cuadrático Medio (RMSE), que penaliza con mayor intensidad las predicciones con errores elevados; y el coeficiente de determinación (R^2 Score), el cual indica qué tan bien el modelo logra explicar la variabilidad presente en los datos.

Posteriormente, el modelo entrenado fue utilizado para realizar una predicción de prueba ingresando las características de un vehículo. Para ello, los datos proporcionados por el usuario son procesados utilizando el mismo procedimiento aplicado durante el entrenamiento, garantizando que la información sea interpretada por la red neuronal bajo las mismas condiciones. Finalmente, la predicción obtenida es transformada nuevamente a su escala original para mostrar un valor monetario comprensible para el usuario.

Con el propósito de facilitar la interpretación del resultado y ofrecer una estimación más útil durante el proceso de compra o venta de un vehículo, la aplicación no solo presenta un precio estimado, sino también un rango de precios calculado a partir del margen de error esperado del modelo. De esta manera, el usuario puede conocer un intervalo razonable dentro del cual debería encontrarse el valor del vehículo, reduciendo el riesgo de aceptar ofertas considerablemente por encima o por debajo del precio esperado. Adicionalmente, la aplicación muestra una estimación de exactitud (MAPE) basada en las métricas obtenidas durante el entrenamiento, proporcionando una referencia del nivel de confianza asociado a la predicción realizada.

Figura 4. Predicción realizada utilizando el modelo entrenado.



La Figura 4 muestra el funcionamiento final de la aplicación una vez concluido el entrenamiento. En esta interfaz el usuario puede ingresar las características de un vehículo y obtener un precio estimado generado por la red neuronal. Además del valor predicho, se presentan indicadores como el MAE, el RMSE, el R² Score y una estimación de exactitud basada en el Error Porcentual Absoluto Medio (MAPE). Esta medida expresa el porcentaje promedio de acierto del modelo respecto al valor real del vehículo, facilitando la interpretación de los resultados por parte del usuario final, por otro lado, se muestra un rango de precios sugerido que permite interpretar la predicción considerando el error esperado del modelo. También se incluye un gráfico de dispersión que compara los valores reales con los valores predichos, permitiendo visualizar de forma rápida el nivel de ajuste alcanzado por el modelo.

5. Evaluación y reflexión (Aspectos éticos y de equidad)

El modelo desarrollado tiene como objetivo proporcionar una estimación del precio de un vehículo usado a partir de información histórica suministrada por el usuario. Sin embargo, la

calidad de las predicciones depende directamente de la calidad y representatividad de los datos utilizados durante el entrenamiento. Si el conjunto de datos contiene precios incorrectos, información desactualizada o una cantidad insuficiente de ejemplos, el modelo aprenderá esos patrones y generará estimaciones poco confiables.

Otro aspecto importante es que el modelo no debe considerarse como un mecanismo para determinar el valor exacto de un vehículo, sino como una herramienta de apoyo para la toma de decisiones. Factores como el estado mecánico, modificaciones realizadas al vehículo, historial de mantenimiento o condiciones particulares del mercado no siempre están representados en el conjunto de datos y, por tanto, pueden influir significativamente en el precio real.

Con el fin de reducir posibles sesgos asociados a diferencias entre países, monedas o mercados de vehículos usados, la aplicación permite que cada usuario entrene el modelo utilizando su propio conjunto de datos. De esta manera, las predicciones se adaptan al contexto específico en el que serán utilizadas, disminuyendo el riesgo de obtener estimaciones poco representativas debido al uso de información perteneciente a otra región.

Finalmente, el sistema presenta un rango estimado de precios en lugar de afirmar que la predicción corresponde al valor real del vehículo. Esta decisión busca comunicar al usuario que toda predicción realizada por una red neuronal posee un margen de error inherente y que el resultado debe interpretarse como una referencia para apoyar el proceso de compra o venta, mas no como un criterio único para tomar una decisión económica inmediata.

6. Conclusión

El desarrollo de este proyecto permitió comprobar que las redes neuronales pueden utilizarse como una herramienta de apoyo para estimar el precio de vehículos usados a partir de

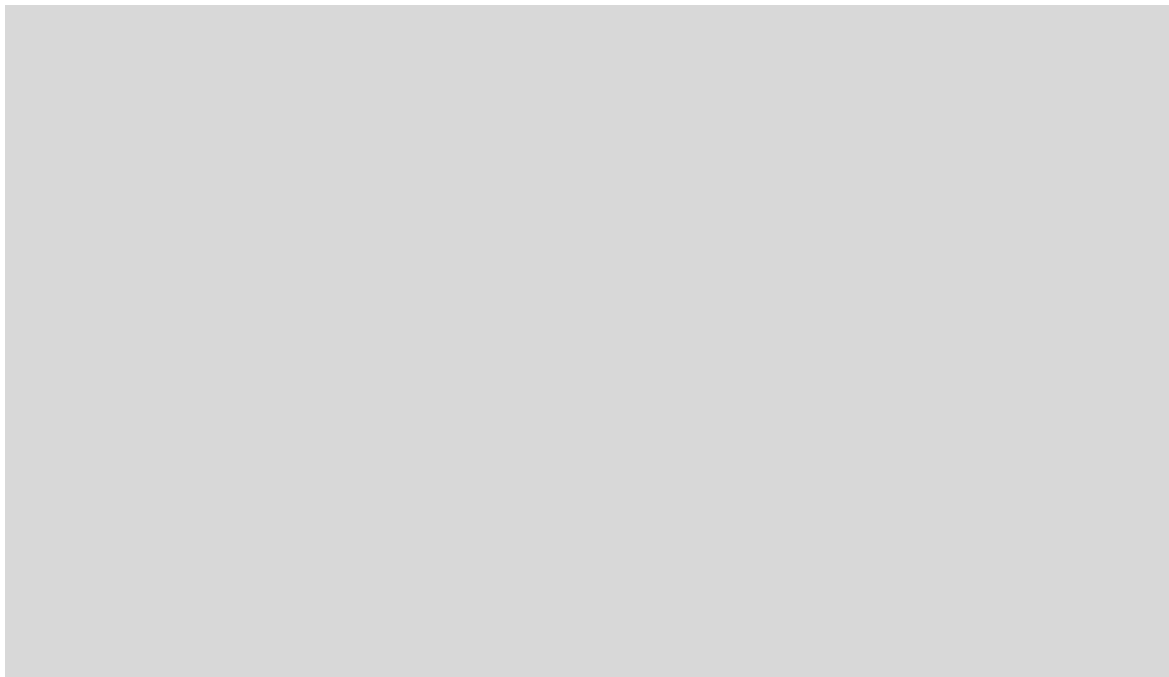
características relevantes como el año, el kilometraje, la marca, el modelo y el historial de accidentes. Los resultados obtenidos demuestran que es posible generar predicciones con un nivel de precisión adecuado, siempre que el modelo sea entrenado con datos representativos y de buena calidad.

No obstante, es importante reconocer que ninguna predicción generada por un modelo de inteligencia artificial debe considerarse como un valor exacto o absoluto. Factores externos que no forman parte del conjunto de datos, como el estado mecánico, la demanda del mercado o las condiciones particulares del vehículo, pueden influir significativamente en su precio real. Por esta razón, la aplicación debe entenderse como una herramienta de apoyo para la toma de decisiones y no como un sustituto del criterio del comprador o del vendedor.

Enlaces

Repositorio: <https://github.com/MiloGonza/AutoPriceANN.git>

Video: <https://www.youtube.com/watch?v=H3UvcHv2KmM>



Referencias

Ribeiro, M. T., Wu, T., Guestrin, C., & Singh, S. (2020). *Beyond accuracy: Behavioral testing of NLP models with CheckList*. arXiv. <https://arxiv.org/abs/2012.07564>

Sarker, I. H. (2021). *Deep Learning: A Comprehensive Overview on Techniques, Taxonomy, Applications and Research Directions*. *SN Computer Science*, 2(6), 420.

<https://doi.org/10.1007/s42979-021-00815-1>

Janiesch, C., Zschech, P., & Heinrich, K. (2021). *Machine learning and deep learning*. *Electronic Markets*, 31, 685–695. <https://doi.org/10.1007/s12525-021-00475-2>

Kingma, D. P., & Ba, J. (2015). *Adam: A Method for Stochastic Optimization*. arXiv. <https://arxiv.org/abs/1412.6980>

He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*. arXiv. <https://arxiv.org/abs/1502.01852>